

DESIGN PATTERNS

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

CODE:

```
class Logger {
    private static Logger instance;
    private Logger() {}
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
    public void log(String message) {
        System.out.println("[LOG]: " + message);
    }
}

public class Main {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("This is the first log message.");
        logger2.log("This is the second log message.");

        System.out.println("\n--- Verification Results ---");
        if (logger1 == logger2) {
            System.out.println("SUCCESS: Both variables point to the same instance!");
        } else {
            System.out.println("FAILURE: Different instances exist.");
        }
    }
}
```

Output

```
[LOG]: This is the first log message.
[LOG]: This is the second log message.

--- Verification Results ---
SUCCESS: Both variables point to the same instance!

=== Code Execution Successful ===
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

CODE:

```
interface Document {
    void open();
    void close();
}

class WordDocument implements Document {
    public void open() { System.out.println("Opening Word Document (.docx)..."); }
    public void close() { System.out.println("Closing Word Document."); }
}

class PdfDocument implements Document {
    public void open() { System.out.println("Opening PDF Document (.pdf)..."); }
    public void close() { System.out.println("Closing PDF Document."); }
}

class ExcelDocument implements Document {
    public void open() { System.out.println("Opening Excel Spreadsheet (.xlsx)..."); }
    public void close() { System.out.println("Closing Excel Spreadsheet."); }
}

abstract class DocumentFactory {
    public abstract Document createDocument();
}

class WordDocumentFactory extends DocumentFactory {
    public Document createDocument() { return new WordDocument(); }
}

class PdfDocumentFactory extends DocumentFactory {
    public Document createDocument() { return new PdfDocument(); }
}

class ExcelDocumentFactory extends DocumentFactory {
    public Document createDocument() { return new ExcelDocument(); }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Document Management System ---\n");

        DocumentFactory wordFactory = new WordDocumentFactory();
        Document wordDoc = wordFactory.createDocument();
        wordDoc.open();
        wordDoc.close();
    }
}
```

```

        System.out.println();

        DocumentFactory pdfFactory = new PdfDocumentFactory();
        Document pdfDoc = pdfFactory.createDocument();
        pdfDoc.open();
        pdfDoc.close();

        System.out.println();

        DocumentFactory excelFactory = new ExcelDocumentFactory();
        Document excelDoc = excelFactory.createDocument();
        excelDoc.open();
        excelDoc.close();
    }
}

```

Output

```

--- Document Management System ---

Opening Word Document (.docx)...
Closing Word Document.

Opening PDF Document (.pdf)...
Closing PDF Document.

Opening Excel Spreadsheet (.xlsx)...
Closing Excel Spreadsheet.

=== Code Execution Successful ===

```

Exercise 3: Implementing the Builder Pattern

Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

CODE:

```

class Computer {
    private final String CPU;
    private final String RAM;
    private final String storage;
    private final String graphicsCard;
    private final boolean isBluetoothEnabled;

    private Computer(Builder builder) {
        this.CPU = builder.CPU;
        this.RAM = builder.RAM;
        this.storage = builder.storage;
        this.graphicsCard = builder.graphicsCard;
        this.isBluetoothEnabled = builder.isBluetoothEnabled;
    }
}

```

```

    }

    @Override
    public String toString() {
        return "Computer Spec -> CPU: " + CPU + " | RAM: " + RAM +
            " | Storage: " + (storage != null ? storage : "None") +
            " | GPU: " + (graphicsCard != null ? graphicsCard : "Integrated") +
            " | Bluetooth: " + isBluetoothEnabled;
    }

    public static class Builder {
        private final String CPU;
        private final String RAM;
        private String storage;
        private String graphicsCard;
        private boolean isBluetoothEnabled;

        public Builder(String CPU, String RAM) {
            this.CPU = CPU;
            this.RAM = RAM;
        }

        public Builder setStorage(String storage) {
            this.storage = storage;
            return this;
        }

        public Builder setGraphicsCard(String graphicsCard) {
            this.graphicsCard = graphicsCard;
            return this;
        }

        public Builder setBluetoothEnabled(boolean isBluetoothEnabled) {
            this.isBluetoothEnabled = isBluetoothEnabled;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Computer standard = new Computer.Builder("Intel i5", "16GB").build();
        Computer premium = new Computer.Builder("Intel i9", "64GB")
            .setStorage("1TB SSD")
            .setGraphicsCard("RTX 4070")
            .setBluetoothEnabled(true)
            .build();

        System.out.println(standard);
        System.out.println(premium);
    }
}

```

```
}
```

Output

```
Computer Spec -> CPU: Intel i5 | RAM: 16GB | Storage: None | GPU: Integrated | Bluetooth: false  
Computer Spec -> CPU: Intel i9 | RAM: 64GB | Storage: 1TB SSD | GPU: RTX 4070 | Bluetooth: true  
  
=== Code Execution Successful ===
```

Exercise 4: Implementing the Adapter Pattern

Scenario:

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

CODE:

```
interface PaymentProcessor {  
    void processPayment(double amount);  
}  
  
class PayPalGateway {  
    public void sendPayment(double totalUSD) {  
        System.out.println("PayPal processed: $" + totalUSD);  
    }  
}  
  
class StripeGateway {  
    public void chargeCustomer(int amountInCents) {  
        System.out.println("Stripe processed: " + amountInCents + " cents");  
    }  
}  
  
class PayPalAdapter implements PaymentProcessor {  
    private PayPalGateway gateway;  
    public PayPalAdapter(PayPalGateway gateway) { this.gateway = gateway; }  
    public void processPayment(double amount) { gateway.sendPayment(amount); }  
}  
  
class StripeAdapter implements PaymentProcessor {  
    private StripeGateway gateway;  
    public StripeAdapter(StripeGateway gateway) { this.gateway = gateway; }  
    public void processPayment(double amount) { gateway.chargeCustomer((int)(amount * 100)); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        PaymentProcessor p1 = new PayPalAdapter(new PayPalGateway());  
        PaymentProcessor p2 = new StripeAdapter(new StripeGateway());  
  
        p1.processPayment(19.50);  
        p2.processPayment(19.50);  
    }  
}
```

```
}
```

Output

```
PayPal processed: $19.5  
Stripe processed: 1950 cents  
  
=== Code Execution Successful ===
```

Exercise 5: Implementing the Decorator Pattern

Scenario:

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

CODE:

```
interface Notifier {  
    void send(String message);  
}  
  
class EmailNotifier implements Notifier {  
    public void send(String message) {  
        System.out.println("Email Out: " + message);  
    }  
}  
  
abstract class NotifierDecorator implements Notifier {  
    protected Notifier wrapper;  
    public NotifierDecorator(Notifier notifier) { this.wrapper = notifier; }  
    public void send(String message) { wrapper.send(message); }  
}  
  
class SMSNotifierDecorator extends NotifierDecorator {  
    public SMSNotifierDecorator(Notifier notifier) { super(notifier); }  
    public void send(String message) {  
        super.send(message);  
        System.out.println("SMS Out: " + message);  
    }  
}  
  
class SlackNotifierDecorator extends NotifierDecorator {  
    public SlackNotifierDecorator(Notifier notifier) { super(notifier); }  
    public void send(String message) {  
        super.send(message);  
        System.out.println("Slack Out: " + message);  
    }  
}
```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Deploying Multi-Channel Notification Stack:");  
        Notifier systemAlert = new SlackNotifierDecorator(new SMSNotifierDecorator(new  
EmailNotifier()));  
        systemAlert.send("System Update Successful.");  
    }  
}
```

Output

```
Deploying Multi-Channel Notification Stack:  
Email Out: System Update Successful.  
SMS Out: System Update Successful.  
Slack Out: System Update Successful.  
  
=== Code Execution Successful ===
```

Exercise 6: Implementing the Proxy Pattern

Scenario:

You are developing an image viewer application that loads images from a remote server. Use the Proxy Pattern to add lazy initialization and caching.

CODE:

```
interface Image {  
    void display();  
}  
  
class RealImage implements Image {  
    private String fileName;  
  
    public RealImage(String fileName) {  
        this.fileName = fileName;  
        loadFromRemoteServer();  
    }  
  
    private void loadFromRemoteServer() {  
        System.out.println("-> [Network Loading]: " + fileName);  
    }  
}
```

```

    public void display() {
        System.out.println("-> [Rendering Graphic]: " + fileName);
    }
}

class ProxyImage implements Image {
    private String fileName;
    private RealImage cacheInstance;

    public ProxyImage(String fileName) {
        this.fileName = fileName;
    }

    public void display() {
        if (cacheInstance == null) {
            cacheInstance = new RealImage(fileName);
        } else {
            System.out.println("-> [Cache Hit]: Serving matching asset from cache.");
        }
        cacheInstance.display();
    }
}

public class Main {
    public static void main(String[] args) {
        Image img = new ProxyImage("dashboard_analytics.png");

        System.out.println("--- Action 1 ---");
        img.display();

        System.out.println("\n--- Action 2 ---");
        img.display();
    }
}

```

Output

```

--- Action 1 ---
-> [Network Loading]: dashboard_analytics.png
-> [Rendering Graphic]: dashboard_analytics.png

--- Action 2 ---
-> [Cache Hit]: Serving matching asset from cache.
-> [Rendering Graphic]: dashboard_analytics.png

=== Code Execution Successful ===

```


Exercise 7: Implementing the Observer Pattern

Scenario:

You are developing a stock market monitoring application where multiple clients need to be notified whenever stock prices change. Use the Observer Pattern to achieve this.

CODE:

```
import java.util.ArrayList;
import java.util.List;

interface Observer { void update(String stock, double price); }
interface Stock { void add(Observer o); void remove(Observer o); void notifyAllObservers(); }

class StockMarket implements Stock {
    private List<Observer> list = new ArrayList<>();
    private String sym; private double prc;

    public void add(Observer o) { list.add(o); }
    public void remove(Observer o) { list.remove(o); }
    public void notifyAllObservers() { for(Observer o : list) o.update(sym, prc); }

    public void updatePrice(String sym, double prc) {
        this.sym = sym; this.prc = prc;
        notifyAllObservers();
    }
}

class MobileApp implements Observer {
    public void update(String s, double p) { System.out.println("Mobile View -> " + s + ": $" + p); }
}

class WebApp implements Observer {
    public void update(String s, double p) { System.out.println("Web Chart -> " + s + ": $" + p); }
}

public class Main {
    public static void main(String[] args) {
        StockMarket market = new StockMarket();
        Observer m = new MobileApp();
        Observer w = new WebApp();

        market.add(m); market.add(w);
        System.out.println("--- Update 1 ---");
        market.updatePrice("GOOG", 142.50);
    }
}
```

Output

```
--- Update 1 ---  
Mobile View -> GOOG: $142.5  
Web Chart -> GOOG: $142.5  
  
=== Code Execution Successful ===
```

Exercise 9: Implementing the Command Pattern

Scenario: You are developing a home automation system where commands can be issued to turn devices on or off. Use the Command Pattern to achieve this.

CODE:

```
class Light {  
    public void turnOn() { System.out.println("Light state: ON"); }  
    public void turnOff() { System.out.println("Light state: OFF"); }  
}  
  
interface Command { void execute(); }  
  
class LightOnCommand implements Command {  
    private Light light;  
    public LightOnCommand(Light light) { this.light = light; }  
    public void execute() { light.turnOn(); }  
}  
  
class LightOffCommand implements Command {  
    private Light light;  
    public LightOffCommand(Light light) { this.light = light; }  
    public void execute() { light.turnOff(); }  
}  
  
class RemoteControl {  
    private Command cmd;  
    public void setCommand(Command cmd) { this.cmd = cmd; }  
    public void pressButton() { cmd.execute(); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Light homeLight = new Light();
```

```
RemoteControl remote = new RemoteControl();

remote.setCommand(new LightOnCommand(homeLight));
remote.pressButton();

remote.setCommand(new LightOffCommand(homeLight));
remote.pressButton();
}
```

Output

```
Light state: ON
Light state: OFF
```

```
=== Code Execution Successful ===
```